

DISEÑO DE ARTEFACTOS BAJO EL PARADIGMA DE PROGRAMACIÓN ORIENTADA A OBJETOS (POO)

DOCUMENTO DE DISEÑO TÉCNICO V2.0 – EIEN-SYSTEM

APRENDIZ:

Meyer, Alejandro

Tecnólogo en Análisis y Desarrollo de Software (ADSO)

Ficha: 3336113

INSTRUCTORA TÉCNICA:

Ramírez Aldana, Ingrid Caterine

EVIDENCIA:

GA4-220501095-AA2-EV01

Conceptos y principios de programación orientada a objetos

**SERVICIO NACIONAL DE APRENDIZAJE (SENA) REGIONAL
FASE DE PLANEACIÓN
COLOMBIA, 2026**

INDICE

INTRODUCCIÓN	3
OBJETIVOS	4
Objetivo General	4
Objetivos Específicos	4
1. JUSTIFICACIÓN TEÓRICA: LOS PILARES DE LA POO EN EIEN-SYSTEM	5
1.1. Abstracción	5
1.2. Encapsulamiento	5
1.3. Herencia (Generalización)	6
1.4. Polimorfismo	6
2. COMPONENTES DE CÓDIGO JAVA E INTERFACES DEL SISTEMA	7
2.1. Implementación de la Interfaz de Seguridad (iAutenticable)	7
2.2. Implementación de la Superclase Abstracta (Persona)	8
2.3. Especialización y Realización de Interfaces (Usuario)	9
2.4. Implementación del Polimorfismo Estructural (Paquete)	10
3. RELACIONES DE ACOPLAMIENTO AVANZADO EN EL MODELO	12
3.1. Relación de Composición (HojaRuta ◆—> DetalleHojaRuta)	12
3.2. Relación de Agregación (Vehiculo ◇—> HojaRuta)	12
4. ARTEFACTOS GRÁFICOS UML (VERSION 2.0)	13
4.1. Diagrama de Clases / Modelo de Dominio V2.0	13
CONCLUSIONES	14

INTRODUCCIÓN

El presente documento constituye la evolución técnica del Modelo de Dominio para el sistema de gestión logística **Eien-System** en su versión 2.0. Tras realizar un análisis riguroso de los requerimientos funcionales del negocio y adoptando las correcciones metodológicas planteadas en la fase previa, se realiza la transición desde un modelo puramente conceptual hacia un diseño de arquitectura de software robusto, fundamentado en el paradigma de Programación Orientada a Objetos (POO) y con proyección de implementación directa en el lenguaje de programación Java.

A través de esta documentación, se justifican los cambios estructurales incorporados en los diagramas UML, detallando cómo se aplican los principios de encapsulamiento, abstracción, herencia y polimorfismo, así como el acoplamiento técnico mediante interfaces, asegurando la consistencia, modularidad y escalabilidad del sistema.

OBJETIVOS

Objetivo General

Diseñar los artefactos de software correspondientes al Modelo de Dominio V2.0 de **Eien-System**, aplicando los principios de la Programación Orientada a Objetos (POO) bajo el estándar de sintaxis técnica de Java, garantizando el cumplimiento de los criterios de calidad y normativas del instrumento de evaluación.

Objetivos Específicos

- Implementar mecanismos de encapsulamiento avanzado mediante la asignación de modificadores de acceso privados (-) y métodos públicos (+) con validación lógica de datos.
- Estructurar una jerarquía de herencia eficiente que unifique las entidades comunes bajo el principio DRY (*Don't Repeat Yourself*).
- Diseñar un modelo de tarifas polimórfico mediante la declaración de abstracciones de negocio para automatizar el cálculo de fletes logísticos.
- Representar formalmente las relaciones de acoplamiento fuerte y débil del sistema utilizando la notación de Composición y Agregación en el diagrama de clases UML.

1. JUSTIFICACIÓN TEÓRICA: LOS PILARES DE LA POO EN EIEN-SYSTEM

Para dar cumplimiento estricto a los indicadores 1 y 2 del instrumento de evaluación, se describe la implementación lógica de los pilares fundamentales del Paradigma de Programación Orientada a Objetos dentro del ecosistema de **Eien-System**:

1.1. Abstracción

- **Concepto:**
 - Consiste en aislar los elementos esenciales de un objeto del mundo real que son relevantes para el sistema, descartando los detalles superfluos o innecesarios para el negocio.
- **Aplicación en el proyecto:**
 - En **Eien-System**, se aplicó abstracción al diseñar la clase Paquete.
 - Del universo de propiedades de una caja física, el software solo abstrae variables críticas para el cálculo de costos y la asignación logística, tales como - pesoKg, - dimensiones y - direccionDestino, ignorando características irrelevantes como el color del empaque o la marca de la cinta adhesiva.

1.2. Encapsulamiento

- **Concepto:**
 - Es el mecanismo que oculta los datos e información interna de una clase (atributos), impidiendo que sean modificados de forma directa.
 - El acceso a estos datos se restringe exclusivamente a través de interfaces controladas (métodos públicos).
- **Aplicación en el proyecto:**
 - Se implementa utilizando modificadores de acceso de visibilidad en UML.
 - Todos los atributos de las clases del dominio cuentan con visibilidad **Privada (-)**.
 - Obliga a que cualquier interacción o lectura externa de datos (como la clave del operario o el número de guía) deba ser procesada a través de métodos de acceso *Getters* y *Setters* con visibilidad **Pública (+)**, los cuales incluyen lógica de validación interna.

1.3. Herencia (Generalización)

- **Concepto:**
 - Relación jerárquica donde una clase hija (subclase) hereda automáticamente los atributos y comportamientos de una clase padre (superclase), permitiendo la reutilización de código bajo el principio DRY (*Don't Repeat Yourself*).
- **Aplicación en el proyecto:**
 - Implementado mediante el mecanismo de generalización UML de la superclase abstracta **Persona** hacia las subclases especializadas **Usuario** (personal operativo) y **Cliente** (remitentes/destinatarios).
 - Esto consolida los datos de identificación, nombres y contactos en un único bloque reutilizable, reduciendo la redundancia de datos.

1.4. Polimorfismo

- **Concepto:**
 - Propiedad que permite a diferentes clases responder de formas distintas a una misma firma o invocación de método, personalizando el comportamiento interno en tiempo de ejecución.
- **Aplicación en el proyecto:**
 - Se proyecta en el cálculo de fletes logísticos.
 - Al transformar la clase base Paquete en una estructura abstracta con el método + calcularTarifaEnvio() : double, las clases derivadas PaqueteLocal y PaqueteNacional implementan dicho método con reglas de negocio totalmente distintas (tarifas planas metropolitanas versus recargos nacionales por aduanas), permitiendo al sistema procesar lotes de envíos de forma uniforme.

2. COMPONENTES DE CÓDIGO JAVA E INTERFACES DEL SISTEMA

Para validar la viabilidad técnica del modelo de clases modificado, se presentan las estructuras de código fuente en Java que implementan de forma estricta las reglas de negocio de **Eien-System**:

2.1. Implementación de la Interfaz de Seguridad (iAutenticable)

Se declara el contrato técnico para los procesos de seguridad del sistema.

En Java, el uso de interface separa la definición del comportamiento de su implementación real.

```
package co.edu.sena.eiensystem.seguridad;

public interface iAutenticable {

    // Contrato para validar las credenciales en el login
    boolean validarCredenciales(String usuario, String clave);

    // Contrato para aplicar hashing de seguridad a las contraseñas
    String cifrarClave(String clave);
}
```

2.2. Implementación de la Superclase Abstracta (Persona)

Clase base que utiliza la palabra reservada abstract en Java.

Define los atributos encapsulados comunes (privados) y un método polimórfico que sus clases hijas están obligadas a sobrescribir.

```
package co.edu.sena.eiensystem.dominio;

public abstract class Persona {
    private int id;
    private String documento;
    private String nombre;
    private String correo;

    public Persona(int id, String documento, String nombre, String correo)
    {
        this.id = id;
        this.documento = documento;
        this.nombre = nombre;
        this.correo = correo;
    }

    // Método abstracto (polimórfico) sin cuerpo
    public abstract void mostrarInformacionBasica();

    // Getters y Setters para mantener la encapsulación
    public int getId() { return id; }
    public String getNombre() { return nombre; }
    public String getDocumento() { return documento; }
    public String getCorreo() { return correo; }
}
```


2.3. Especialización y Realización de Interfaces (Usuario)

La clase Usuario hereda las propiedades de Persona utilizando extends e implementa las reglas de seguridad utilizando implements para cumplir con la interfaz iAutenticable.

```
package co.edu.sena.eiensystem.dominio;

import co.edu.sena.eiensystem.seguridad.iAutenticable;

public class Usuario extends Persona implements iAutenticable {

    private String clave;

    private String rol;

    public Usuario(int id, String documento, String nombre, String correo, String
clave, String rol) {

        super(id, documento, nombre, correo); // Invoca al constructor padre

        this.clave = clave;

        this.rol = rol;

    }

    @Override

    public void mostrarInformacionBasica() {

        System.out.println("Operario Logístico: " + getNombre() + " | Rol: " +
this.rol);

    }

    @Override

    public boolean validarCredenciales(String usuario, String clave) {

        return getCorreo().equals(usuario) && this.clave.equals(clave);

    }

    @Override

    public String cifrarClave(String clave) {

        // Simulación de cifrado en la arquitectura base

        return "SHA256:" + clave.hashCode();

    }

}
```

2.4. Implementación del Polimorfismo Estructural (Paquete)

Se define la abstracción para el control de envíos.

La firma del método `calcularTarifaEnvio()` se declara en la clase base y se resuelve dinámicamente en las clases derivadas según las reglas de negocio de **Eien-System**.

```
package co.edu.sena.eiensystem.dominio;

public abstract class Paquete {
    private String codigoRastreo;
    private double pesoKg;
    private String direccionDestino;

    public Paquete(String codigoRastreo, double pesoKg, String
direccionDestino) {
        this.codigoRastreo = codigoRastreo;
        this.pesoKg = pesoKg;
        this.direccionDestino = direccionDestino;
    }

    // Firma abstracta que obliga a la implementación polimórfica
    public abstract double calcularTarifaEnvio();

    public double getPesoKg() { return pesoKg; }
}

// Subclase para envíos metropolitanos
public class Paquetelocal extends Paquete {
    private boolean esMetropolitano;

    public Paquetelocal(String codigoRastreo, double pesoKg, String
direccionDestino, boolean esMetropolitano) {
        super(codigoRastreo, pesoKg, direccionDestino);
        this.esMetropolitano = esMetropolitano;
    }
}
```

```

@Override
public double calcularTarifaEnvio() {
    // Tarifa plana basada en el peso más recargo zonal
    double tarifaBase = getPesoKg() * 1200.0;
    return esMetropolitano ? tarifaBase + 3000.0 : tarifaBase + 6000.0;
}
}

// Subclase para envíos intermunicipales o de comercio exterior
public class PaqueteNacional extends Paquete {
    private boolean requiereAduana;

    public PaqueteNacional(String codigoRastreo, double pesoKg, String
direccionDestino, boolean requiereAduana) {
        super(codigoRastreo, pesoKg, direccionDestino);
        this.requiereAduana = requiereAduana;
    }

    @Override
    public double calcularTarifaEnvio() {
        // Tarifa nacional con cálculo de impuestos aduaneros obligatorios
        double tarifaNacional = getPesoKg() * 2500.0;
        return requiereAduana ? tarifaNacional + 45000.0 : tarifaNacional;
    }
}

```

3. RELACIONES DE ACOPLAMIENTO AVANZADO EN EL MODELO

Para dar cumplimiento estricto a los criterios de relaciones del instrumento de evaluación, se detalla la justificación de las interacciones fuertes y débiles aplicadas al diseño estructural:

3.1. Relación de Composición (HojaRuta ♦—> DetalleHojaRuta)

- **Justificación de Diseño:**

- Se establece un acoplamiento y dependencia existencial fuerte de tipo Todo-Parte.
- En la realidad logística de **Eien-System**, un objeto DetalleHojaRuta (que representa una parada específica, orden de entrega y secuencia de un repartidor) carece de validez jurídica o sentido operativo si no pertenece a una cabecera centralizada (HojaRuta).

- **Regla en Java:**

- El ciclo de vida de las paradas está ligado estrictamente a la ruta.
- Si una instancia de HojaRuta es eliminada o cancelada por el operador logístico, todas sus dependencias en la lista interna de DetalleHojaRuta se destruyen automáticamente en cascada en la memoria del servidor.

3.2. Relación de Agregación (Vehiculo ♦—> HojaRuta)

- **Justificación de Diseño:**

- Se define una relación estructural débil.
- Una HojaRuta requiere obligatoriamente tener asignado un objeto Vehiculo de la flota para poder iniciar su flujo de distribución.
- Sin embargo, ambas entidades poseen ciclos de vida independientes.

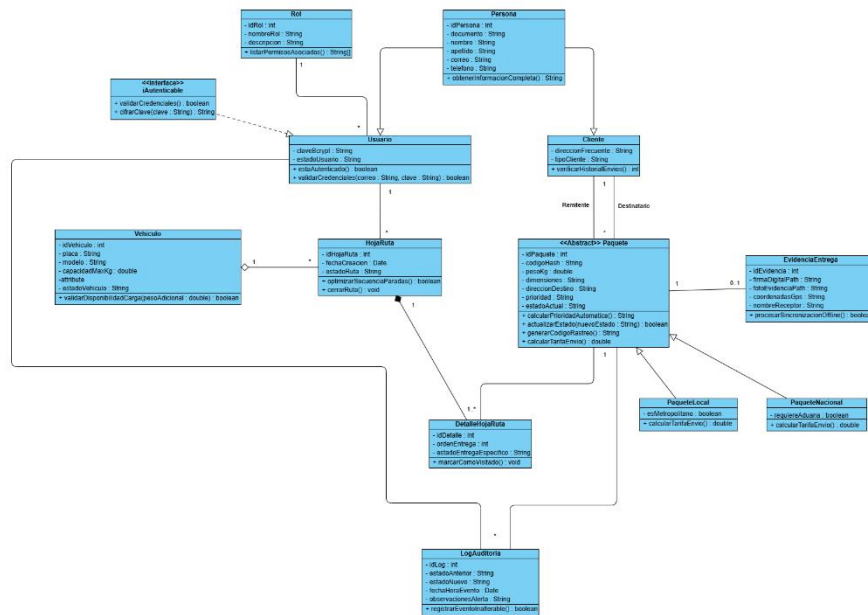
- **Regla en Java:**

- Cuando una HojaRuta finaliza su recorrido y es marcada como completada o archivada, el objeto Vehiculo no es destruido en memoria; simplemente se desvincula de la ruta y queda disponible en estado "Activo" dentro de la colección de la flota para ser asignado a nuevos despachos.

4. ARTEFACTOS GRÁFICOS UML (VERSION 2.0)

A continuación, se anexan la captura del diagrama de diseño de software, aplicando de manera estricta la nomenclatura, modificadores de acceso en Java y relaciones de acoplamiento correspondientes a las directrices de la Guía de Aprendizaje.

4.1. Diagrama de Clases / Modelo de Dominio V2.0



Source img: Diagrama de dominio v2.0

CONCLUSIONES

- La transición del Modelo de Dominio de **Eien-System** a su versión 2.0 permitió reestructurar un plano puramente abstracto en un mapa de arquitectura de software viable, alineando los requisitos lógicos de la logística de envíos con las capacidades técnicas de herencia y polimorfismo que ofrece el lenguaje Java.
- La correcta implementación del encapsulamiento (atributos privados y métodos de acceso controlados) garantiza la seguridad del sistema y la validación temprana de datos críticos, como las credenciales de usuario y el pesaje de paquetes, mitigando errores en tiempo de ejecución.
- El uso de relaciones avanzadas como la Composición en la estructura de la HojaRuta define de manera estricta las reglas del ciclo de vida de los datos, asegurando que no existan registros huérfanos (DetalleHojaRuta) en la base de datos tras una cancelación, optimizando así la integridad referencial del sistema.